Instituto Tecnológico de Veracruz

Ingeniería en Sistemas Computacionales

Materia: Administración en Base de Datos

Unidad 3

Creación y Configuración de un Clúster

Reporte de Investigación y Práctica de Laboratorio

Alumno

Raúl Alberto Guerrero Aguilar

Índice general

Haga clic sobre cualquier entrada para saltar a la sección correspondiente.

1. Introducción

2. Objetivos

2.1. Objetivo general
2.2. Objetivos específicos

3. Contenido general / Marco teórico

3.1. Arquitectura de un clúster en MongoDB: Replica Sets
3.2. Configuración y dispersión del espacio en disco: Sharded Clusters
3.3. Motor de almacenamiento WiredTiger
3.4. Protocolo de conexión mongodb+srv://

4. Desarrollo de la práctica (paso a paso)

4.1. Paso 1: Acceso al proyecto e inicio de creación del clúster
4.2. Paso 2: Configuración del clúster, proveedor cloud, región y selección del tier
4.3. Paso 3: Configuración de seguridad de red (IP Whitelist) y usuario administrador
4.4. Paso 4: Selección del método de conexión y extracción de la cadena URI
4.5. Paso 5: Conexión mediante código Python (PyMongo) e inserción de documentos
4.5.1. Preparación del entorno e instalación del controlador
4.5.2. Código de MongoDBConnections.py
4.5.3. Operación interna, resultado y verificación
4.6. Paso 6: Instalación y conexión por terminal mediante MongoDB Shell (mongosh)
4.6.1. Instalación de mongosh en Windows
4.6.2. Comando de conexión y autenticación
4.6.3. Interpretación del prompt
4.6.4. Comandos ejecutados en la sesión

5. Tabla comparativa técnica: driver (PyMongo) vs. terminal (mongosh) vs. interfaz web (Data Explorer)

6. Conclusión

Índice de figuras

Índice de las 11 figuras del reporte.

Figura	Descripción	Ubicación
Figura 1	Pantalla inicial de Project 0 Overview en la organización Raúl Alberto's Org haciendo clic en el botón + Create.	Paso 1
Figura 2	Pantalla Deploy your cluster mostrando las especificaciones de los planes M10, Flex y Free, el proveedor AWS y la región Mexico (mx-central-1).	Paso 2
Figura 3	Ventana Connect to ClusterPractica en su estado inicial solicitando la dirección IP y la creación del primer usuario de base de datos.	Paso 3
Figura 4	Registro de la dirección IP 187.190.156.164 y configuración de las credenciales del usuario AdminPractica.	Paso 3
Figura 5	Menú de selección de métodos de conexión (Drivers and Client Libraries, AI Tools, Atlas SQL, Compass, Shell y MongoDB for VS Code).	Paso 4
Figura 6	Ventana Connecting with MongoDB Client Library mostrando la cadena de conexión mongodb+srv://AdminPractica:<db_password>@clusterpractica.gbs3rjy.mongodb.net/.	Paso 4
Figura 7	Panel general de Project 0 Overview confirmando el estado ACTIVE de ClusterPractica (Tier FREE, MongoDB v8.0.34, 0 B / 512 MB).	Paso 4
Figura 8	Entorno PyCharm ejecutando MongoDBConnections.py y mostrando en consola: ¡Documento insertado con éxito! ID asignado: 6ad6d7714cb690c6e7afacd1.	Paso 5
Figura 9	Herramienta Data Explorer de MongoDB Atlas mostrando la base de datos mi_primera_bd, la colección usuarios y el documento insertado con el ObjectId('6ad6d7714cb690c6e7afacd1').	Paso 5
Figura 10	Ventana del asistente de instalación MongoDB Shell Setup Wizard en Windows.	Paso 6
Figura 11	Terminal CMD de Windows conectada al nodo primario del clúster mediante mongosh v2.13.0, mostrando la salida de show dbs (mi_primera_bd 40.00 KiB), use mi_primera_bd y db.usuarios.find().	Paso 6

1. Introducción

Las bases de datos NoSQL surgieron como respuesta a las limitaciones que el modelo relacional presenta ante cargas de trabajo masivas, esquemas cambiantes y la necesidad de escalar horizontalmente sobre hardware comercial. Dentro de esta familia, las bases de datos **documentales** almacenan la información en documentos autocontenidos, jerárquicos y de estructura flexible, de modo que una entidad completa (por ejemplo, un usuario con sus atributos anidados) puede leerse o escribirse en una sola operación sin recurrir a uniones costosas. MongoDB es el representante más difundido de este paradigma: serializa sus documentos en formato **BSON** (*Binary JSON*), una extensión binaria de JSON que añade tipos como `ObjectId`, fechas, enteros de 32 y 64 bits y datos binarios.

Para ofrecer disponibilidad continua y capacidad ilimitada, MongoDB no se despliega normalmente como una instancia aislada, sino como un **clúster distribuido**: un conjunto de servidores que cooperan para mantener copias sincronizadas de los datos (*Replica Sets*) y, cuando el volumen excede la capacidad de un solo equipo, para repartir las colecciones entre varios grupos de servidores (*Sharded Clusters*). Operar este tipo de infraestructura de forma manual exige editar archivos de configuración, generar llaves de autenticación entre nodos, inicializar el conjunto de réplicas y vigilar permanentemente el estado del almacenamiento.

El paradigma **DBaaS** (*Database-as-a-Service*) traslada esa carga operativa al proveedor. **MongoDB Atlas** es el servicio administrado oficial de MongoDB y automatiza el aprovisionamiento, la replicación, el cifrado en tránsito, las actualizaciones de versión y los respaldos sobre la infraestructura de Amazon Web Services, Google Cloud o Microsoft Azure. El administrador interactúa con el clúster mediante una consola web y cadenas de conexión estándar, lo que permite concentrarse en el diseño de datos y de aplicaciones.

El presente reporte documenta una práctica de laboratorio de la Unidad 3 en la que se aprovisionó el clúster `ClusterPractica` dentro de la organización `Raúl Alberto's Org` y el proyecto `Project 0`; se evaluaron los niveles de servicio (*tiers*) M10, Flex y Free; se configuraron las dos capas de seguridad obligatorias (lista de acceso IP y usuario administrador); y se estableció una **conectividad bimodal**: una conexión programática mediante el controlador `PyMongo` desde Python y una conexión interactiva mediante `MongoDB Shell (mongosh)` desde la terminal de Windows. Ambos canales se contrastaron entre sí y con la herramienta gráfica *Data Explorer* de Atlas. El documento se organiza en objetivos, marco teórico, desarrollo paso a paso con once evidencias, tabla comparativa y conclusiones.

2. Objetivos

2.1. Objetivo general

Aprovisionar un clúster NoSQL en MongoDB Atlas, comprender los mecanismos internos de replicación, distribución y almacenamiento que lo sustentan, y verificar su operación mediante dos formas de conectividad independientes (código Python y terminal `mongosh`) sobre una misma base de datos documental.

2.2. Objetivos específicos

- **Objetivo específico 1.** Describir la arquitectura de un clúster MongoDB basada en *Replica Sets* (nodo primario, secundarios, *Oplog* y *failover* automático) y contrastarla con la configuración manual mediante `/etc/mongod.conf` y `rs.initiate()`.
- **Objetivo específico 2.** Explicar la dispersión del espacio en disco mediante *Sharded Clusters*, incluyendo `mongos`, *Config Servers*, *Shards*, *Shard Keys*, *Chunks* y el *Balancer*.
- **Objetivo específico 3.** Analizar el motor de almacenamiento *WiredTiger* en cuanto a caché, compresión BSON, reutilización de bloques eliminados, compactación y *Storage Auto-scaling*, y evaluar los tiers M10, Flex y Free para desplegar `ClusterPractica`.
- **Objetivo específico 4.** Configurar la seguridad del clúster mediante la lista de acceso IP y la creación del usuario administrador `AdminPractica` con permisos `atlasAdmin`.
- **Objetivo específico 5.** Establecer, ejecutar y comparar la conexión por `PyMongo` y por `mongosh` mediante el protocolo `mongodb+srv://`, validando la inserción de un documento con el *Data Explorer* de Atlas.

3. Contenido general / Marco teórico

3.1. Arquitectura de un clúster en MongoDB: Replica Sets

Un **Replica Set** es un conjunto de procesos `mongod` que mantienen exactamente el mismo conjunto de datos. Su finalidad es doble: proporcionar **redundancia** (si un disco o servidor se pierde, existen copias completas en otros equipos) y **alta disponibilidad** (el servicio continúa sin intervención humana ante la caída de un nodo). Se recomienda un mínimo de tres miembros con derecho a voto para que siempre exista una mayoría capaz de elegir un nuevo primario.

- **Nodo primario (*Primary*).** Es el único miembro que acepta operaciones de escritura de los clientes, ya sean scripts de Python o comandos de terminal. Cada escritura aplicada se registra en el *Oplog* (*Operations Log*), una colección de tamaño fijo (`local.oplog.rs`) donde las operaciones se almacenan de forma idempotente y en orden.
- **Nodos secundarios (*Secondaries*).** Leen de forma asíncrona el *Oplog* del primario y reproducen sus operaciones sobre sus propias copias, manteniendo colecciones idénticas en sus discos. Pueden atender lecturas si el cliente configura una *read preference* que lo permita.
- **Conmutación por error automática (*Automatic Failover*).** Los miembros intercambian señales de vida (*heartbeats*) cada 2 segundos. Si el primario deja de responder durante el tiempo de espera de elección (10 segundos por defecto), los secundarios convocan una elección basada en un protocolo de consenso tipo Raft; el candidato que obtiene la mayoría de votos es promovido a nuevo primario. Los controladores modernos reintentan automáticamente ciertas escrituras (*retryable writes*) para que la aplicación sea transparente al evento.

En una instalación **manual**, el administrador debe instalar `mongod` en cada servidor, editar el archivo `/etc/mongod.conf` para declarar el nombre del conjunto de réplicas, la interfaz de red y la autenticación, crear un archivo `keyFile` compartido para autenticar a los miembros entre sí y, finalmente, ejecutar `rs.initiate()` y `rs.add()` desde un cliente.

```
# /etc/mongod.conf  (fragmento ilustrativo de instalación manual)
replication:
  replSetName: "rs0"
net:
  port: 27017
  bindIp: 0.0.0.0
security:
  keyFile: /etc/mongo/keyfile
  authorization: enabled

// Desde mongosh, en uno de los nodos:
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "nodo1:27017" },
    { _id: 1, host: "nodo2:27017" },
    { _id: 2, host: "nodo3:27017" }
  ]
})
```

Tabla 1. Configuración manual de un Replica Set frente al aprovisionamiento en MongoDB Atlas.

Aspecto	Instalación manual	MongoDB Atlas (DBaaS)
Infraestructura	El administrador adquiere, instala y mantiene los servidores.	Atlas aprovisiona las máquinas en AWS, Google Cloud o Azure.
Configuración	Edición manual de `/etc/mongod.conf` en cada nodo.	Asistente web; los parámetros se generan automáticamente.
Autenticación entre nodos	Creación y distribución de un `keyFile` o certificados x.509.	Gestionada internamente por la plataforma.
Inicialización del conjunto	Ejecución de `rs.initiate()` y `rs.add()`.	Automática; el clúster nace con tres nodos replicados.
Cifrado en tránsito	Requiere generar y rotar certificados TLS.	TLS/SSL habilitado por defecto.
Parches y versiones	Actualización manual nodo por nodo.	Actualizaciones administradas por Atlas.

3.2. Configuración y dispersión del espacio en disco: Sharded Clusters

Cuando el volumen de datos o la carga de operaciones exceden la capacidad de un solo servidor, MongoDB recurre al **Sharding**, una forma de escalabilidad horizontal que fragmenta las colecciones y las distribuye entre varios servidores. Cada servidor conserva solo una porción de los datos, por lo que el espacio total de almacenamiento crece al añadir *shards* en lugar de ampliar un único disco. Es importante precisar que el *sharding* en Atlas está disponible a partir de los clústeres dedicados de gama alta; los tiers evaluados en esta práctica (M10, Flex y Free) operan como un único *Replica Set* y no lo utilizan, por lo que esta sección tiene carácter conceptual.

Tabla 2. Componentes de un Sharded Cluster.

Componente	Función
Query Routers (`mongos`)	Enrutadores de consultas. La aplicación se conecta a ellos y no directamente a los datos; cada `mongos` consulta los metadatos y redirige la petición al *shard* que contiene el dato solicitado.
Config Servers	Conjunto de réplicas que almacena los metadatos del clúster y el mapa exacto que indica en qué servidor físico reside cada bloque (*chunk*) de datos.
Shards	Cada *shard* es un *Replica Set* independiente que guarda una fracción de los datos totales y hereda, por tanto, la redundancia y el *failover* descritos en la sección anterior.

La distribución se rige por la **Shard Key**, un campo (o combinación de campos) indexado de cada documento que MongoDB utiliza para decidir a qué fragmento pertenece. Existen dos estrategias principales:

Tabla 3. Estrategias de Shard Key: por rango frente a hashed.

Característica	Por rango (*ranged*)	Hashed
Funcionamiento	Los documentos se agrupan en intervalos contiguos de valores de la clave.	Se aplica una función hash al valor de la clave y se reparte según el resultado.
Consultas por intervalo	Muy eficientes: se dirigen a pocos *shards*.	Ineficientes: suelen consultar todos los *shards*.
Distribución de escrituras	Riesgo de *hot spot* si la clave crece de forma monótona (por ejemplo, fechas u `ObjectId`).	Distribución uniforme de las inserciones entre los *shards*.
Uso recomendado	Claves con buena cardinalidad y consultas por rango.	Cargas de escritura intensivas con claves monótonas.

Sobre la Shard Key, MongoDB divide los datos en bloques lógicos denominados **Chunks**. El tamaño de *chunk* de **64 MB** fue el valor predeterminado histórico del sistema; a partir de la versión 6.0 el valor por defecto es de 128 MB y es configurable, pero el concepto es idéntico: cuando un *chunk* excede el límite se divide en dos. Un proceso en segundo plano llamado **Balancer** monitorea la cantidad de *chunks* de cada *shard* y migra bloques entre ellos hasta que todos los discos mantengan un porcentaje de ocupación equilibrado, evitando que un servidor se llene mientras otros permanecen vacíos.

3.3. Motor de almacenamiento WiredTiger

En cada servidor físico, el motor **WiredTiger** (motor por defecto desde MongoDB 3.2) gestiona cómo se escriben los documentos en el disco y cómo se utilizan los recursos de memoria. Sus características principales son las siguientes:

- **Caché en RAM.** Reserva por defecto el mayor valor entre el 50 % de (RAM − 1 GB) y 256 MB para mantener en memoria los índices y los documentos de uso frecuente. El resto de la memoria queda disponible para el sistema operativo, cuya caché de archivos también acelera las lecturas.
- **Compresión en disco.** Los datos se almacenan en formato BSON y se comprimen por bloque con el algoritmo *Snappy* (por defecto) o *zstd* (mayor razón de compresión a costa de CPU); los índices utilizan compresión por prefijos. En conjunto, la compresión reduce el espacio en disco aproximadamente entre un 50 % y un 70 %.
- **Concurrencia a nivel de documento.** Dos operaciones pueden modificar documentos distintos de la misma colección simultáneamente, sin bloqueos de colección completa.
- **Durabilidad.** Combina un registro de escritura anticipada (*journal*) con puntos de control (*checkpoints*) periódicos, que consolidan en disco una imagen consistente de los datos.

- **Reutilización de espacio.** Al eliminar documentos con `deleteMany()`, WiredTiger no reduce el tamaño del archivo físico en el sistema operativo; marca los bloques liberados como reutilizables para futuras inserciones. En servidores locales el espacio puede devolverse al sistema operativo con `db.runCommand({ compact: "coleccion" })`, mientras que en MongoDB Atlas el espacio se administra mediante el escalado del clúster.

En Atlas, los clústeres dedicados incorporan **Storage Auto-scaling**: cuando el uso de disco alcanza un umbral, la plataforma amplía automáticamente el volumen sin interrumpir el servicio. En contraste, el tier Free (M0) cuenta con una cuota fija de 512 MB que no escala.

Tabla 4. Parámetros de gestión de almacenamiento de WiredTiger.

Parámetro	Comportamiento	Implicación operativa
Caché interna	50 % de (RAM − 1 GB), mínimo 256 MB.	Dimensionar la RAM según el tamaño del conjunto de trabajo (índices y documentos activos).
Algoritmo de compresión	*Snappy* (por defecto) o *zstd*.	Ahorro de 50 %–70 % de disco; *zstd* comprime más pero consume más CPU.
Borrado de documentos	Los bloques se marcan como libres, sin reducir el archivo.	El espacio se reutiliza en inserciones posteriores.
Recuperación de espacio	`compact` en instalaciones locales.	En Atlas se gestiona mediante el escalado del clúster.
Auto-scaling de almacenamiento	Aplica a clústeres dedicados de Atlas.	No disponible en M0, que mantiene 512 MB fijos.

3.4. Protocolo de conexión mongodb+srv://

El esquema `mongodb+srv://` es una variante de la cadena de conexión estándar que delega la localización del clúster al sistema de nombres de dominio. En lugar de enumerar manualmente la dirección de cada nodo, el cliente realiza una consulta DNS de tipo **SRV** (RFC 2782) al nombre `_mongodb._tcp.clusterpractica.gbs3rjy.mongodb.net`, que devuelve los nombres de host y puertos de todos los miembros del *Replica Set*. Una consulta **TXT** complementaria aporta opciones como `authSource` y `replicaSet`.

A partir de la lista obtenida, el controlador abre conexiones, monitorea la topología mediante los *heartbeats* y determina cuál miembro es el **nodo primario**, dirigiendo hacia él todas las escrituras. Si ocurre un *failover*, el controlador redescubre la topología sin que la aplicación cambie su cadena de conexión. Además, el esquema SRV activa por defecto el cifrado **TLS/SSL**, de modo que las credenciales y los datos viajan protegidos entre el cliente y Atlas.

Tabla 5. Anatomía de la cadena de conexión SRV utilizada en la práctica.

Elemento	Valor en la práctica	Función
Esquema	`mongodb+srv://`	Indica descubrimiento por DNS SRV y TLS por defecto.
Credenciales	`AdminPractica:PracticaUnidad3`	Usuario y contraseña del usuario de base de datos.
Host (dominio SRV)	`clusterpractica.gbs3rjy.mongodb.net`	Dominio cuyos registros SRV/TXT describen el clúster.
Ruta	`/`	Sin base de datos predeterminada; se selecciona desde el cliente.

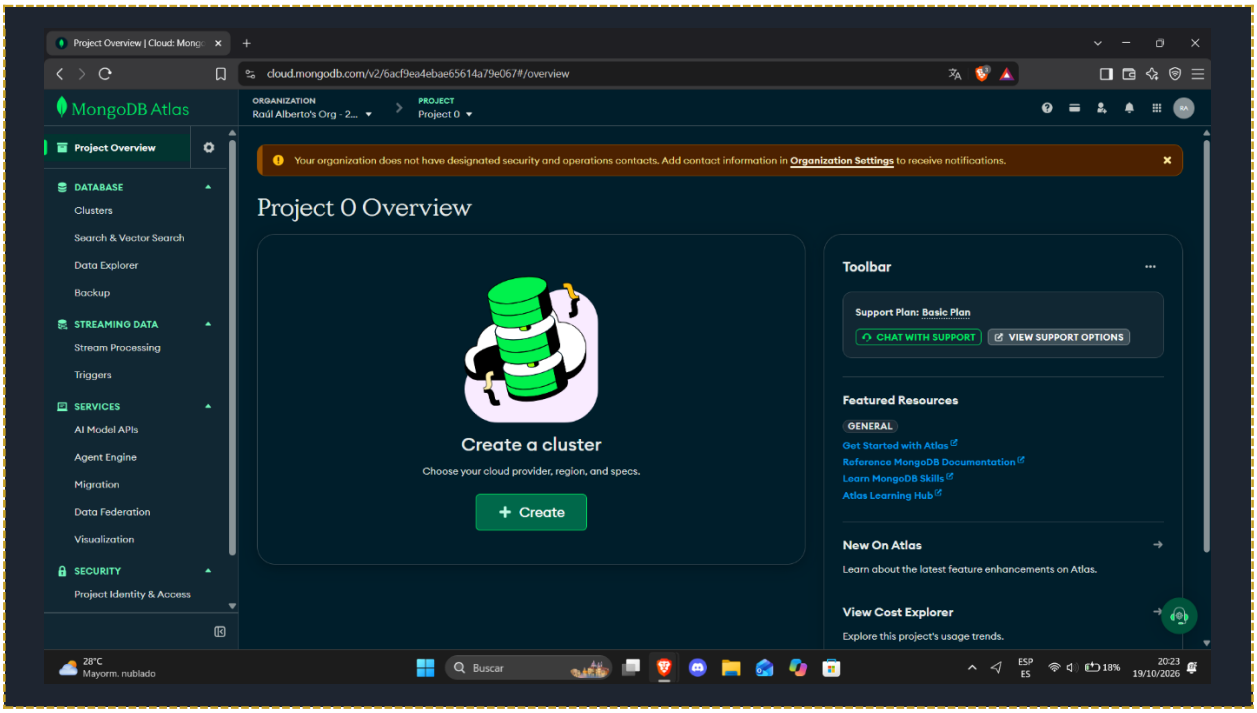
4. Desarrollo de la práctica (paso a paso)

A continuación, se describe el flujo de trabajo completo. En cada paso se indica el espacio reservado para la evidencia fotográfica correspondiente; las capturas están numeradas de la 1 a la 11 siguiendo el orden en que se realizó la práctica.

4.1. Paso 1: Acceso al proyecto e inicio de creación del clúster

MongoDB Atlas organiza sus recursos en una jerarquía de tres niveles: la **organización** (entidad de facturación y de gestión de accesos), el **proyecto** (agrupa clústeres y comparte reglas de red y usuarios) y el **clúster** propiamente dicho. Tras iniciar sesión en la consola web, se ingresó a la organización `Raúl Alberto's Org` y al proyecto `Project 0`.

En la pantalla *Project 0 Overview*, que muestra los recursos del proyecto (inicialmente vacío), se hizo clic en el botón `+ Create`, el cual inicia el asistente de aprovisionamiento de un nuevo clúster.



4.2. Paso 2: Configuración del clúster, proveedor cloud, región y selección del tier

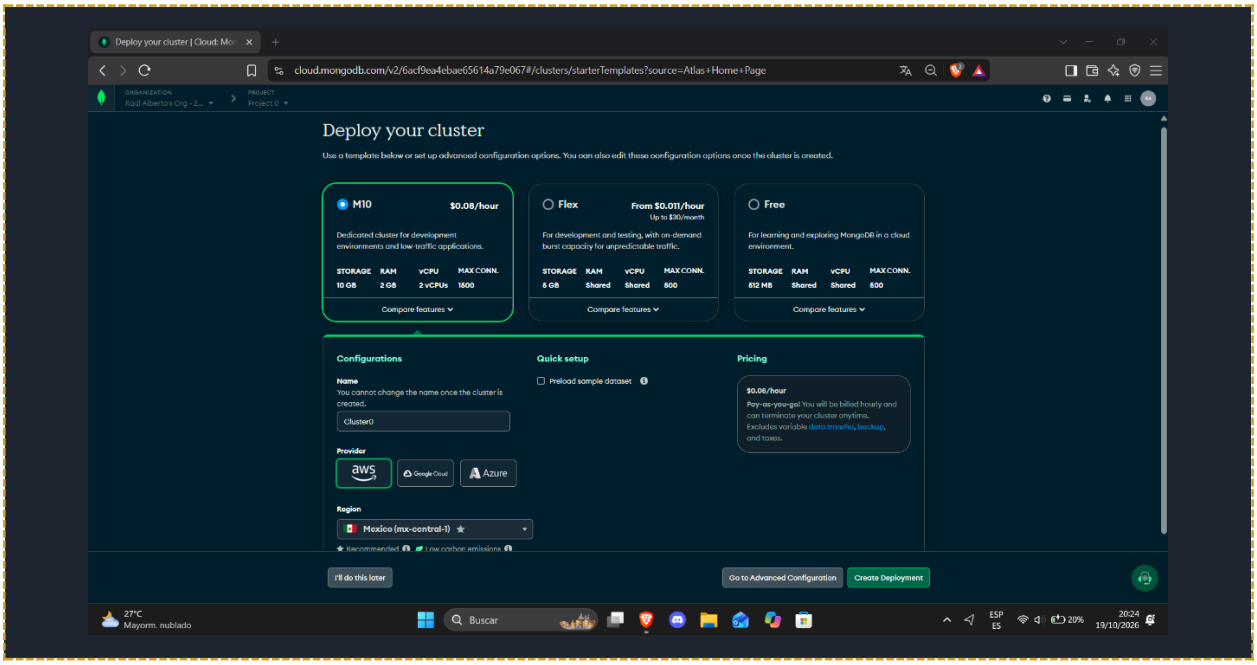
En la pantalla *Deploy your cluster* se presentan las plantillas de despliegue disponibles. Se evaluaron tres niveles de servicio, resumidos en la Tabla 6. Inicialmente se preseleccionó el plan **M10** sobre el proveedor **AWS** (Amazon Web Services) en la región `Mexico (mx-central-1)`, por tratarse de un clúster dedicado con 2 GB de RAM, 2 vCPUs y 10 GB de almacenamiento, a un costo de $0.08 USD por hora (pago por uso).

Tabla 6. Comparativa de los tiers evaluados en MongoDB Atlas.

Parámetro	Tier M10 (Dedicated)	Tier Flex	Tier Free (M0)
Caso de uso	Entornos de desarrollo dedicados y bajo tráfico.	Desarrollo y pruebas con tráfico impredecible.	Aprendizaje y exploración de MongoDB en la nube.
Costo operativo	$0.08 USD/hora (pago por uso).	Desde $0.011 USD/hora (hasta $30/mes).	$0.00 USD (gratuito).
Almacenamiento	10 GB iniciales (auto-scaling hasta 4 TB).	5 GB.	512 MB fijos.
Memoria RAM	2 GB dedicados.	Compartida.	Compartida.
Procesador (vCPU)	2 vCPUs dedicados.	Compartido.	Compartido.
Conexiones simultáneas	1,500 máximo.	500 máximo.	500 máximo.

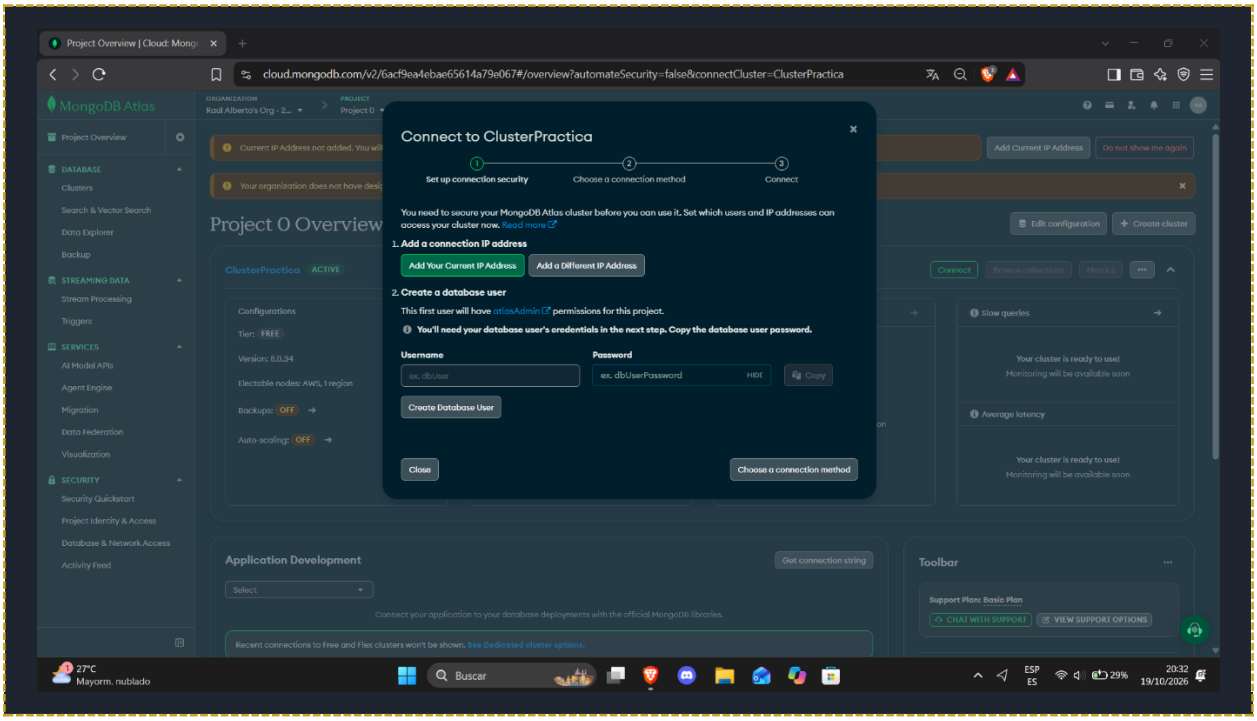
Sin embargo, Atlas solicita obligatoriamente el registro de una tarjeta bancaria como método de validación antifraude para habilitar los clústeres de pago por uso, **independientemente** de que la cuenta cuente con un cupón de crédito de $50 USD. Al no vincularse una tarjeta, no fue posible activar los planes de pago; por ello se optó por desplegar el clúster `ClusterPractica` en el tier **FREE (M0)**, que no requiere método de pago, con la versión **8.0.34** de MongoDB sobre **AWS** (1 región).

Esta decisión impone limitaciones que deben tenerse presentes: el almacenamiento queda acotado a 512 MB, los recursos de CPU y RAM son compartidos con otros clústeres gratuitos, no existe *auto-scaling* de disco y el límite de conexiones simultáneas es de 500. Para los fines didácticos de la práctica (insertar y consultar un documento) dichas restricciones resultan suficientes.



4.3. Paso 3: Configuración de seguridad de red (IP Whitelist) y usuario administrador

Una vez creado el clúster, Atlas abre el asistente *Connect to ClusterPractica*, que exige dos configuraciones de seguridad antes de permitir cualquier conexión. En su estado inicial, la ventana solicita la dirección IP desde la cual se conectará el cliente y la creación del primer usuario de base de datos.



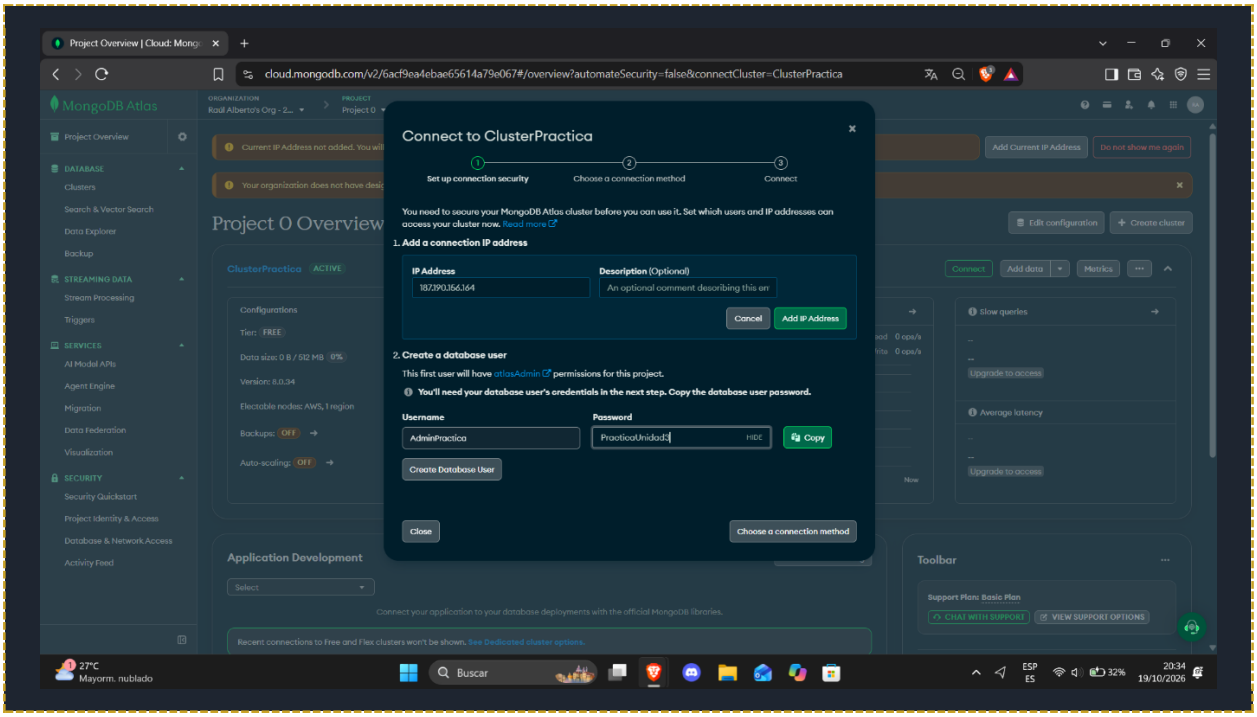
Lista de acceso IP (firewall de red). Por seguridad, Atlas bloquea por defecto todo el tráfico entrante hacia el clúster (política de denegación implícita). Mediante el botón *Add IP Address* se registró la dirección IP pública del equipo cliente, `187.190.156.164`, autorizando exclusivamente las peticiones originadas en ella. Esta práctica evita abrir el clúster a todo Internet (`0.0.0.0/0`), opción que Atlas desaconseja salvo para pruebas muy específicas.

Creación del usuario administrador (RBAC). Atlas emplea control de acceso basado en roles (*Role-Based Access Control*). Mediante el botón *Create Database User* se creó el usuario `AdminPractica`, con la contraseña `PracticaUnidad3`, asignándole el rol `atlasAdmin` sobre el proyecto, que concede privilegios completos de lectura, escritura y administración sobre las bases de datos del clúster.

Tabla 7. Parámetros de seguridad configurados en ClusterPractica.

Capa de seguridad	Configuración aplicada
Control de red (IP Access List)	Dirección IP autorizada: `187.190.156.164`.
Autenticación	Usuario `AdminPractica` con contraseña `PracticaUnidad3`.
Autorización (RBAC)	Rol `atlasAdmin`.
Cifrado en tránsito	TLS/SSL activado por el esquema `mongodb+srv://`.

Cabe señalar que las credenciales se muestran en este documento únicamente por tratarse de una práctica académica sobre un clúster gratuito. En un entorno real, las contraseñas no deben incrustarse en el código fuente ni publicarse en reportes, sino almacenarse en variables de entorno o en un gestor de secretos.

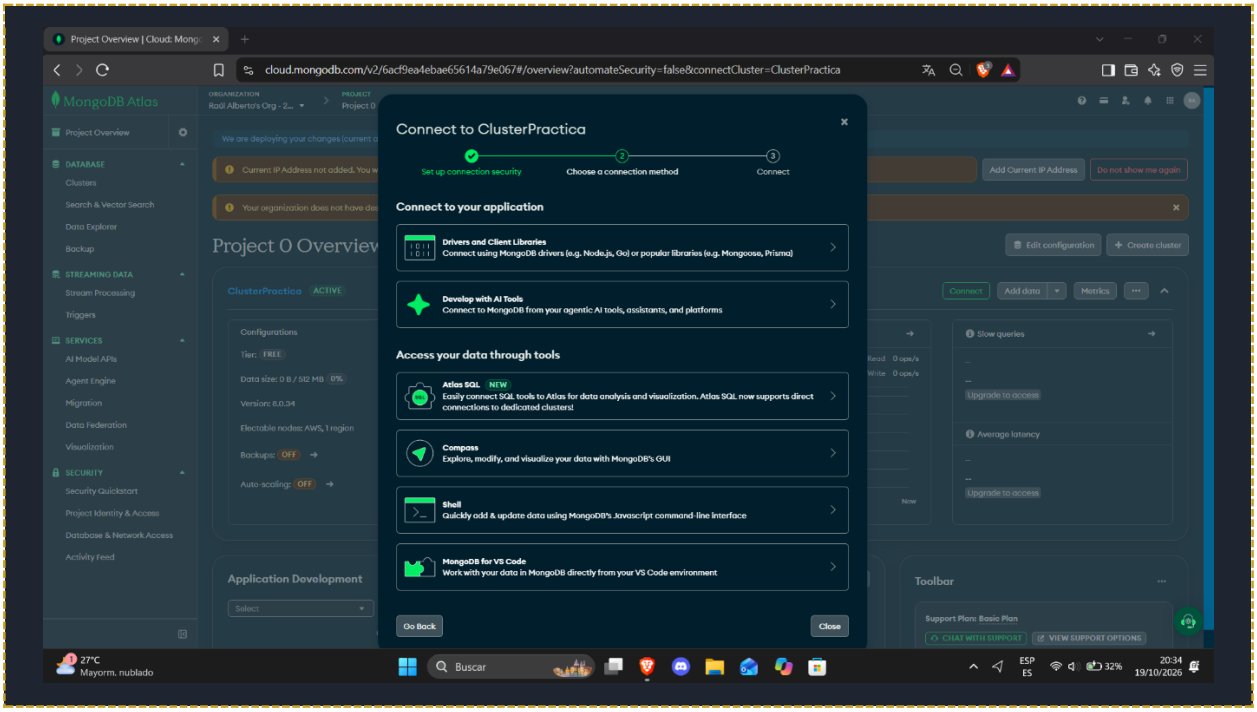


4.4. Paso 4: Selección del método de conexión y extracción de la cadena URI

Concluida la configuración de seguridad, el asistente presenta los métodos de conexión que Atlas ofrece. Cada uno atiende un escenario distinto de uso:

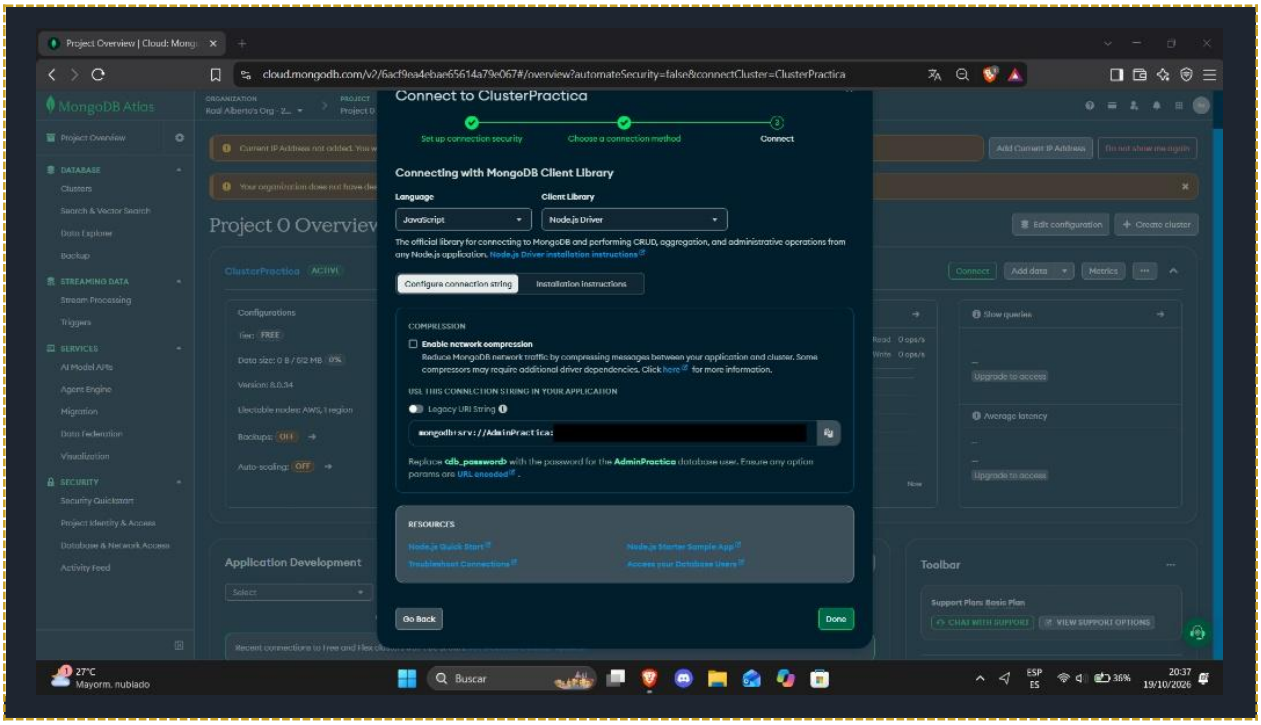
Tabla 8. Métodos de conexión ofrecidos por el asistente de Atlas.

Método	Descripción
Drivers and Client Libraries	Cadena URI para integrar el clúster en aplicaciones mediante controladores oficiales (Python, Node.js, Java, etc.).
AI Tools	Integraciones para conectar el clúster con herramientas y agentes de inteligencia artificial.
Atlas SQL	Interfaz que permite consultar los datos mediante SQL desde herramientas de inteligencia de negocios.
Compass	Cliente gráfico de escritorio para explorar y consultar colecciones.
Shell	Acceso mediante `MongoDB Shell (mongosh)` desde la línea de comandos.
MongoDB for VS Code	Extensión de Visual Studio Code para explorar el clúster y ejecutar operaciones desde el editor.

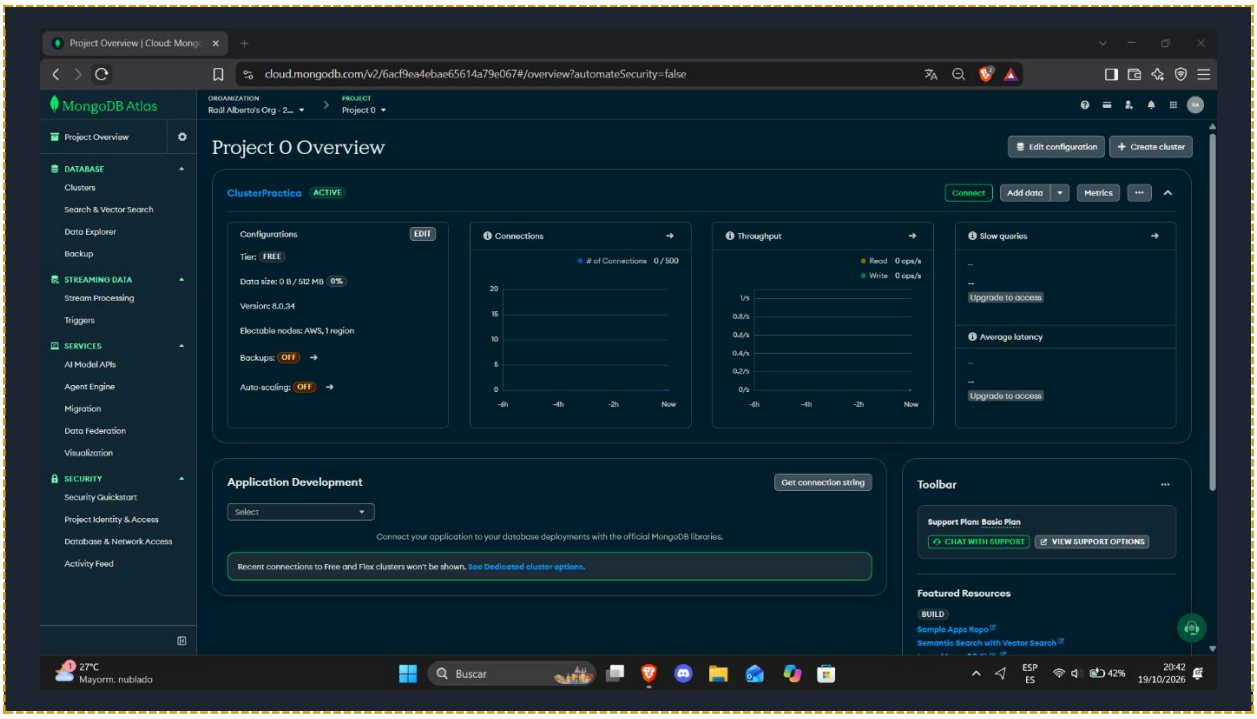


Para la conexión programática se seleccionó **Drivers and Client Libraries**, opción que despliega la ventana *Connecting with MongoDB Client Library*. Allí Atlas entrega la cadena de conexión con un marcador de posición para la contraseña: `mongodb+srv://AdminPractica:<db_password>@clusterpractica.gbs3rjy.mongodb.net/`.

El marcador `<db_password>` debe sustituirse por la contraseña real (`PracticaUnidad3`) antes de usar la cadena; si una contraseña incluyera caracteres especiales, estos tendrían que codificarse en formato porcentual (*percent-encoding*).



Finalmente, al cerrar el asistente se regresó al panel *Project 0 Overview*, donde se confirmó que `ClusterPractica` alcanzó el estado **ACTIVE**, con tier FREE, versión de MongoDB 8.0.34 y un uso de almacenamiento de 0 B sobre la cuota de 512 MB, lo que indica que el clúster se encuentra operativo y vacío.



4.5. Paso 5: Conexión mediante código Python (PyMongo) e inserción de documentos

4.5.1. Preparación del entorno e instalación del controlador

Se utilizó el IDE **PyCharm** con el proyecto `WelcomeScreen`, que cuenta con un entorno virtual `.venv` basado en Python 3.14.8. Dentro de dicho entorno se instaló el controlador oficial de MongoDB con soporte para registros SRV, indispensable para resolver cadenas `mongodb+srv://`:

```
pip install "pymongo[srv]"
```

El extra `[srv]` instala la dependencia `dnspython`, la cual permite al controlador efectuar las consultas DNS SRV y TXT descritas en el marco teórico.

4.5.2. Código de MongoDBConnections.py

En el archivo `MongoDBConnections.py` se implementó el siguiente programa, que se conecta al clúster, crea una base de datos y una colección, e inserta un documento de prueba:

```
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi

uri = "mongodb+srv://AdminPractica:PracticaUnidad3@clusterpractica.gbs3rjy.mongodb.net/"

# Conectar al clúster fijando la versión 1 del Server API
client = MongoClient(uri, server_api=ServerApi('1'))

try:
    # Creación diferida (Lazy Creation) de base de datos y colección
    db = client['mi_primera_bd']
    coleccion = db['usuarios']

    # Inserción de documento de prueba
    nuevo_usuario = {
        "nombre": "Raúl",
        "rol": "Admin",
        "proyecto": "Unidad 3"
    }
    resultado = coleccion.insert_one(nuevo_usuario)

    print(f"¡Documento insertado con éxito! ID asignado: {resultado.inserted_id}")
except Exception as e:
    print(f"Error al conectar o insertar: {e}")
```

Tabla 9. Explicación línea por línea de MongoDBConnections.py.

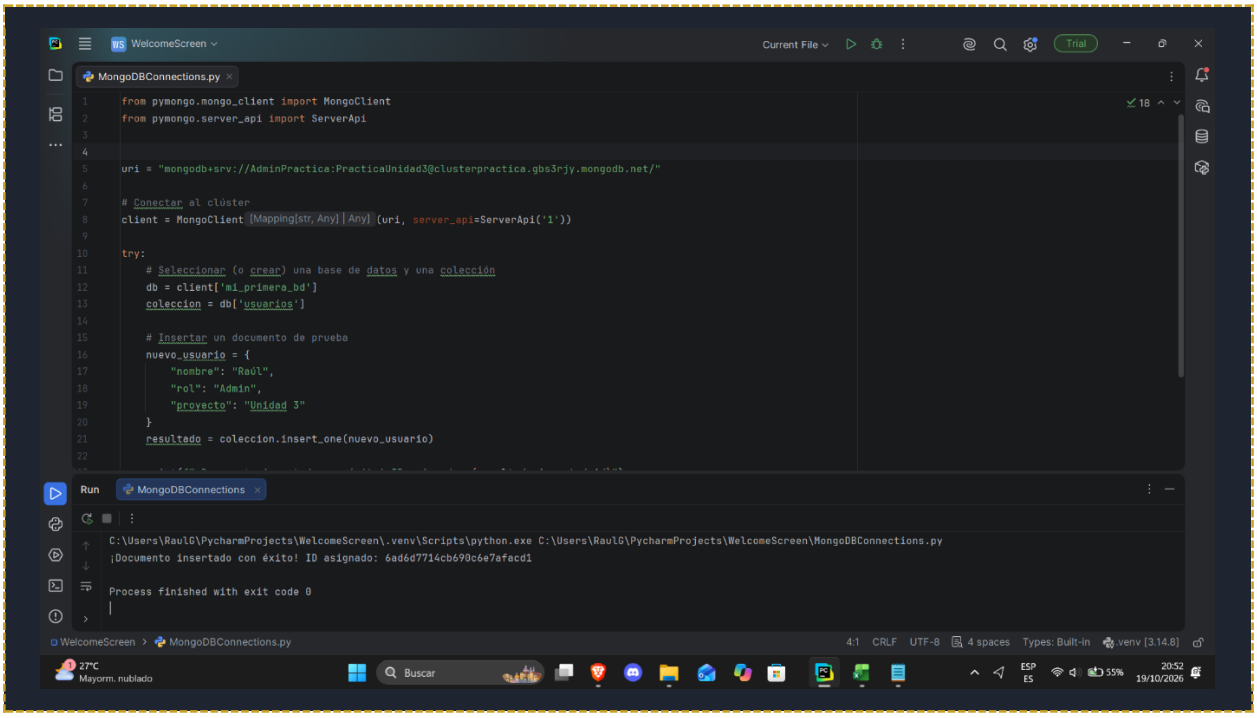
Fragmento de código	Explicación
`from pymongo.mongo_client import MongoClient`	Importa la clase que representa la conexión con el clúster y administra internamente un *connection pool*.
`from pymongo.server_api import ServerApi`	Importa la clase que permite declarar la versión de la API del servidor a la que se desea adherir el cliente.
`uri = "mongodb+srv://..."`	Define la cadena de conexión SRV con usuario, contraseña y dominio del clúster obtenida en el Paso 4.
`client = MongoClient(uri, server_api=ServerApi('1'))`	Crea el cliente. Con `ServerApi('1')` se fija la API versión 1, garantizando un comportamiento estable ante futuras versiones del servidor. La conexión real se establece de forma perezosa en la primera operación.
`try: ... except Exception as e:`	Bloque de manejo de errores: captura fallos de red, autenticación o lista de acceso IP y los muestra en consola.
`db = client['mi_primera_bd']`	Obtiene una referencia a la base de datos. Aún no existe físicamente en el servidor.
`coleccion = db['usuarios']`	Obtiene una referencia a la colección `usuarios`, tampoco creada todavía.
`nuevo_usuario = {...}`	Diccionario de Python con los campos `nombre`, `rol` y `proyecto`; el controlador lo serializa a BSON.

Fragmento de código	Explicación
`resultado = coleccion.insert_one(nuevo_usuario)`	Envía la escritura al nodo primario. En este instante MongoDB crea la base de datos y la colección, y asigna al documento un `_id` de tipo `ObjectId`.
`print(f"... {resultado.inserted_id}")`	Imprime el identificador único generado por el clúster como confirmación.

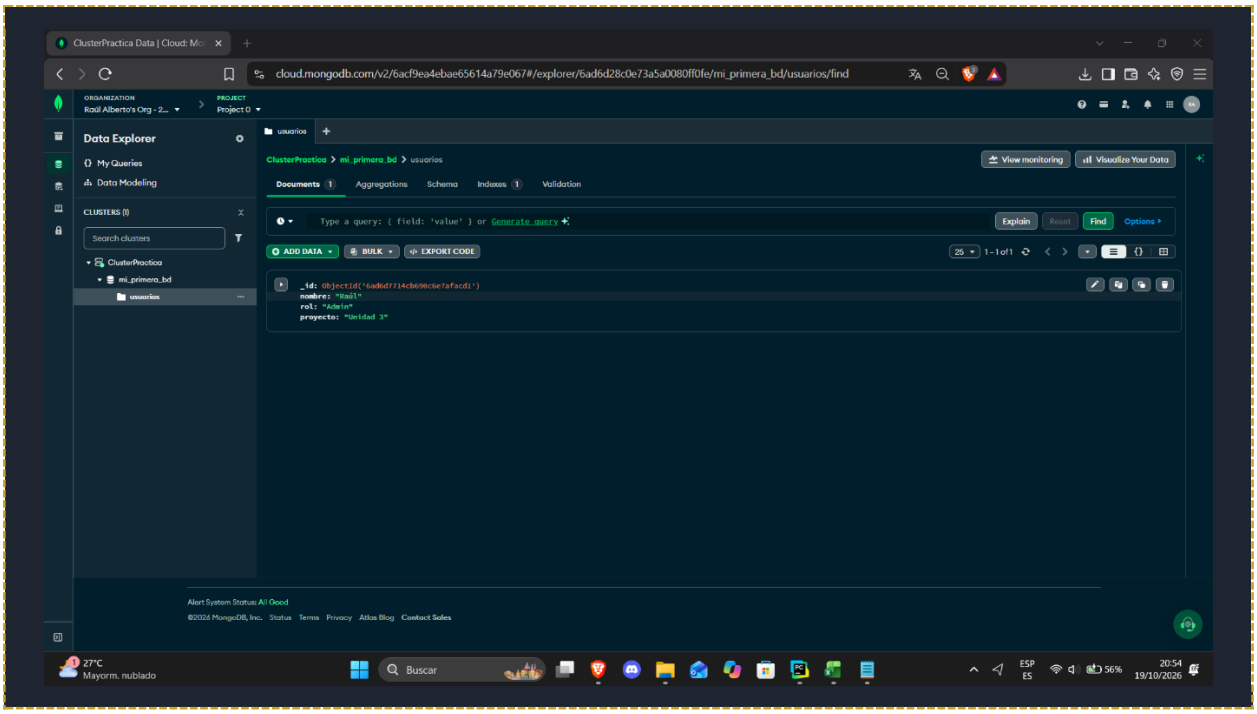
4.5.3. Operación interna, resultado y verificación

MongoDB aplica el principio de **creación diferida** (*Lazy Creation*): la base de datos `mi_primera_bd` y la colección `usuarios` no se crean cuando se declaran en las variables de Python, sino en el instante exacto en que se ejecuta `insert_one()`. El clúster recibió el diccionario, le asignó el identificador único `_id: ObjectId('6ad6d7714cb690c6e7afacd1')` y devolvió la confirmación, que se visualizó en la consola de PyCharm:

```
¡Documento insertado con éxito! ID asignado: 6ad6d7714cb690c6e7afacd1
```



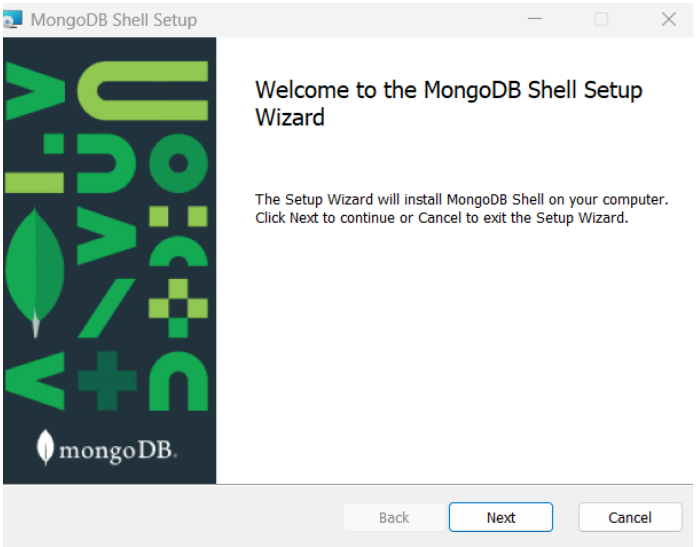
Posteriormente se verificó, mediante la interfaz web **Data Explorer** de Atlas en el navegador, que el documento quedó almacenado correctamente en la colección `usuarios` de la base de datos `mi_primera_bd`, con los campos `nombre: "Raúl"`, `rol: "Admin"` y `proyecto: "Unidad 3"`. Esta comprobación independiente confirma que la escritura fue aceptada por el nodo primario y persistida en el clúster.



4.6. Paso 6: Instalación y conexión por terminal mediante MongoDB Shell (mongosh)

4.6.1. Instalación de mongosh en Windows

Para interactuar directamente con el motor de base de datos desde la línea de comandos se utilizó **MongoDB Shell (`mongosh`)**, un intérprete interactivo basado en JavaScript/Node.js. Se ejecutó el instalador oficial **MongoDB Shell Setup Wizard** (paquete `.msi`) en Windows, instalando la versión **2.13.0** y registrando el ejecutable `mongosh` en la variable de entorno `PATH` del sistema, lo que permite invocarlo desde cualquier directorio del símbolo del sistema.



4.6.2. Comando de conexión y autenticación

Desde el **Símbolo del sistema (CMD)** de Windows se ejecutó el siguiente comando de conexión:

```
mongosh "mongodb+srv://clusterpractica.gbs3rjy.mongodb.net/" --apiVersion 1 --username AdminPractica
```

Obsérvese que la contraseña no se incluye en la línea de comandos: al omitirla, `mongosh` la solicita de forma interactiva, evitando que quede registrada en el historial del terminal. Tras introducir la contraseña `PracticaUnidad3`, la terminal estableció sesión con MongoDB `8.0.34` y mostró el siguiente prompt:

```
Atlas atlas-3f7de5-shard-0 [primary] test>
```

4.6.3. Interpretación del prompt

Tabla 10. Significado de cada elemento del prompt de mongosh.

Elemento	Significado
`Atlas`	La sesión está conectada a un clúster administrado por MongoDB Atlas.
`atlas-3f7de5-shard-0`	Nombre del *Replica Set* al que pertenece la conexión (el sufijo `shard-0` es la convención de nombres de Atlas).
`[primary]`	Confirma que la sesión se conectó directamente al nodo primario del *Replica Set*, el único que acepta escrituras.
`test>`	Base de datos activa por defecto al iniciar la sesión.

4.6.4. Comandos ejecutados en la sesión

Tabla 11. Comandos ejecutados en mongosh y su resultado.

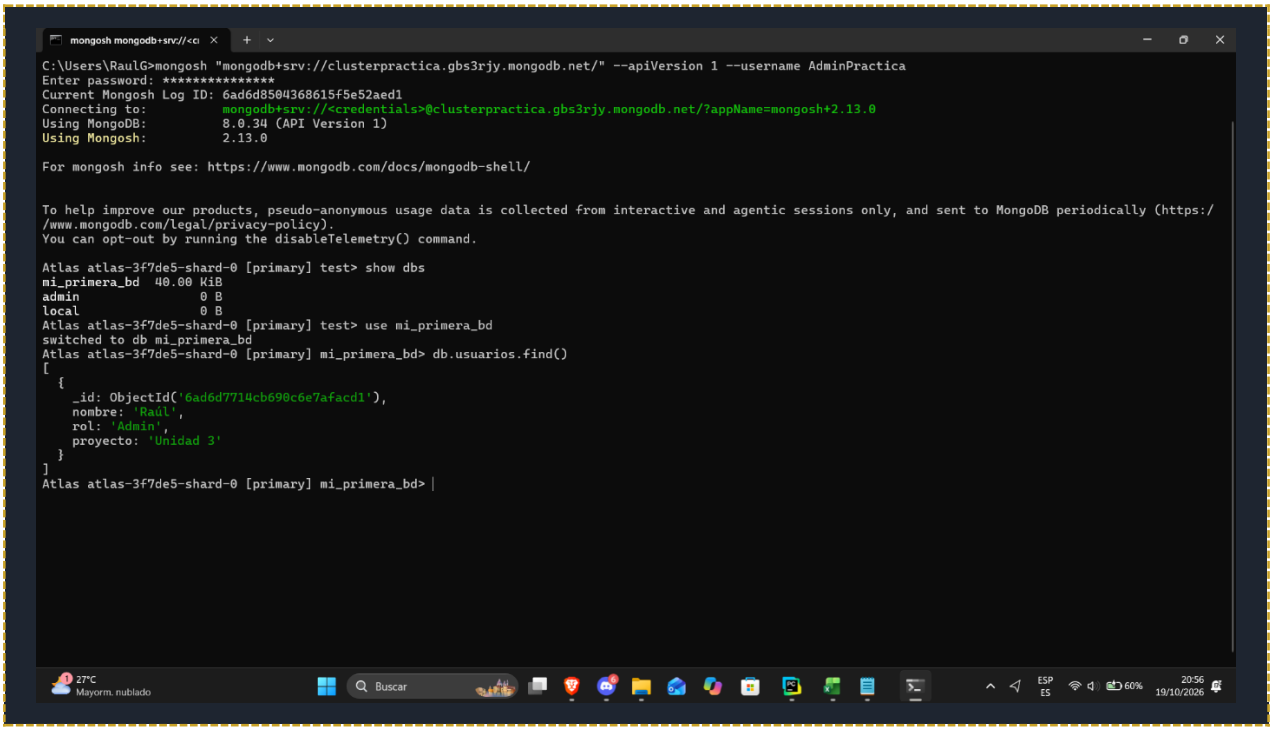
Comando	Resultado y análisis
`show dbs`	Mostró las bases de datos activas y el espacio físico comprimido en disco utilizado por cada una: `mi_primera_bd` con **40.00 KiB**, `admin` con 0 B y `local` con 0 B. El tamaño de `mi_primera_bd` refleja la compresión WiredTiger y las estructuras mínimas de la colección y del índice `_id`.
`use mi_primera_bd`	Cambió el contexto activo de la consola hacia la base de datos creada previamente desde Python.
`db.usuarios.find()`	Consultó todos los documentos de la colección `usuarios`, devolviendo en formato BSON el registro insertado desde Python.

```
Atlas atlas-3f7de5-shard-0 [primary] test> show dbs
mi_primera_bd  40.00 KiB
admin             0 B
local             0 B

Atlas atlas-3f7de5-shard-0 [primary] test> use mi_primera_bd
switched to db mi_primera_bd

Atlas atlas-3f7de5-shard-0 [primary] mi_primera_bd> db.usuarios.find()
[
  {
    _id: ObjectId('6ad6d7714cb690c6e7afacd1'),
    nombre: 'Raúl',
    rol: 'Admin',
    proyecto: 'Unidad 3'
  }
]
```

El documento recuperado coincide exactamente, incluido el `ObjectId`, con el insertado desde Python, lo que demuestra que ambos canales de conexión operan sobre el mismo nodo primario y los mismos datos.



5. Tabla comparativa técnica: driver (PyMongo) vs. terminal (mongosh) vs. interfaz web (Data Explorer)

Los tres mecanismos utilizados en la práctica acceden al mismo clúster, pero difieren en su entorno de ejecución, propósito y modelo de conexión. La Tabla siguiente sintetiza dichas diferencias:

Tabla 12. Comparativa técnica entre PyMongo, mongosh y Data Explorer.

Criterio	Driver Python (`PyMongo`)	`MongoDB Shell (mongosh)`	Atlas `Data Explorer`
Entorno de ejecución	Integrado en el código fuente del backend o script (`.py`).	Consola interactiva de línea de comandos (CMD / PowerShell / Bash).	Navegador web dentro del portal de MongoDB Atlas.
Propósito principal	Automatizar operaciones CRUD desde una aplicación de software.	Administración directa del clúster, auditoría de espacio (`show dbs`) y consultas rápidas.	Inspección visual de colecciones, documentos e índices sin escribir código.
Formato de datos	Diccionarios nativos de Python (`dict`) que el driver serializa a BSON.	Objetos JavaScript / BSON nativos evaluados en tiempo real.	Representación visual de objetos BSON e interfaz de filtros GUI.
Manejo de conexión	Gestiona un *Connection Pool* persistente para múltiples hilos de aplicación.	Abre una sesión interactiva única ligada a la ventana de terminal.	Realiza peticiones HTTPS internas desde el panel de control de Atlas.
Autenticación	Credenciales dentro de la URI (idealmente desde variables de entorno).	Usuario por parámetro y contraseña solicitada de forma interactiva.	Sesión del portal de Atlas del propietario del proyecto.
Mejor uso	Aplicaciones en producción y procesos automatizados.	Administración, diagnóstico y pruebas puntuales.	Verificación visual y consulta ocasional.

6. Conclusión

La práctica permitió comprobar que MongoDB Atlas reduce de manera significativa la complejidad operativa de un clúster NoSQL. Tareas que en una instalación manual implican editar `/etc/mongod.conf`, distribuir un `keyFile` y ejecutar `rs.initiate()` se resolvieron mediante un asistente web que entregó un *Replica Set* de tres nodos, con *failover* automático y cifrado TLS, listo para recibir conexiones. La evidencia del prompt `Atlas atlas-3f7de5-shard-0 [primary]` confirma que el cliente se enlazó al nodo primario, el único autorizado para escribir.

En cuanto al aprovisionamiento, se evaluaron los tiers M10, Flex y Free. Aunque el M10 en AWS (región `Mexico (mx-central-1)`) resultaba el más adecuado por sus recursos dedicados, la exigencia de vincular una tarjeta bancaria —incluso contando con un cupón de $50 USD— llevó a desplegar `ClusterPractica` en el tier Free (M0), con MongoDB 8.0.34 y 512 MB de almacenamiento fijo. Esta limitación subraya la importancia de dimensionar el clúster según las necesidades reales y de considerar las características de escalado, ya que M0 carece de *Storage Auto-scaling*.

En lo relativo a la gestión del almacenamiento, el análisis del motor *WiredTiger* explicó el comportamiento observado: el comando `show dbs` reportó 40.00 KiB para `mi_primera_bd`, cifra coherente con la compresión por bloques y con el tamaño mínimo de una colección con un solo documento y su índice `_id`. Asimismo, se estudió que el borrado de documentos no libera espacio físico de inmediato sino que lo deja disponible para reutilización, y que el *sharding* —con `mongos`, *Config Servers*, *Shard Keys*, *Chunks* y *Balancer*— constituye el mecanismo de dispersión de disco cuando un solo *Replica Set* resulta insuficiente.

Respecto a la conectividad, las dos modalidades resultaron complementarias. `PyMongo` es la vía idónea para integrar el clúster en aplicaciones, pues gestiona un *connection pool* y serializa diccionarios de Python a BSON; `mongosh` es la herramienta adecuada para administración y auditoría inmediata; y el *Data Explorer* ofrece verificación visual sin código. Ambas conexiones, basadas en `mongodb+srv://`, aprovecharon la resolución DNS SRV para descubrir automáticamente el nodo primario, y la consulta final en `mongosh` recuperó el mismo documento (`ObjectId('6ad6d7714cb690c6e7afacd1')`) insertado desde Python, validando la consistencia de los datos. Con ello se cumplieron los cinco objetivos específicos planteados.